

```
//
//  main.cpp
//  IK
//
//  Created by Hyun Joon Shin on 2021/06/09.
//
```

```
#include <iostream>
#define JGL2_IMPLEMENTATION
#include <JGL2/JGL.hpp>
#include <JGL2/Simulation3DView.hpp>
#include <filesystem>
#include <fstream>
#include "j3a.hpp"
#include "quat.hpp"
```

```
using namespace JGL2;
using namespace std;
using namespace jm;
```

```
Simulation3DView<JR::PBRRenderer>* animView;
const vec3 G = vec3(0, -980, 0);
const float DEF_KS = 20000;
const float DEF_KD = 1;
const float k_drag = 1;
```

```
struct Sphere {
    vec3 p;
    float r;
    float k = 0.2;

    Sphere(const vec3& pp, float rr) : p(pp), r(rr) {}
    void draw() const {
        JR::drawSphere(p, r, vec4(.2, .5, .8, 1));
    }
};
```

```
struct Particle {
    vec3 x, v;
    vec3 f;
    float m;
    bool pinned = false;
    Particle(float mass, const vec3& p, const vec3& vel = vec3(0, 0, 0)) :
        x(p), v(vel), m(mass), f(0) {}
    void clearForce() { f = vec3(0); }
    void addForce(const vec3 force) { f += force; }
    void update(float dt) {
        if (!pinned) {
            v += f / m * dt;
            x += v * dt;
        }
        else v = vec3(0);
    }
    void draw() const {
        JR::drawSphere(x, .3, vec4(1, .6, 0, 1));
    }

    void resolveCollision(const Sphere& s) {
        vec3 N = normalize(x - s.p);
        float dist = length(x - s.p) - s.r;
        if (abs(dist) < 0.001 && abs(dot(v, N)) < 0.001) {
            v = v - dot(v, N) * N;
            x = x - dist * N;
        }
        else if (dist < 0.001 && dot(v, N) < 0) {
            vec3 vN = dot(v, N) * N;
            v = v - vN - s.k * vN;
            x = x - dist * N;
        }
    }
};
```

```
struct Spring {
    float r;
    float ks, kd;
    Particle &a, &b;
    Spring(Particle& aa, Particle& bb, float kss = DEF_KS, float kds = DEF_KD)
        : a(aa), b(bb), ks(kss), kd(kds), r(length(aa.x - bb.x)) {}
    void applyForce() {
        vec3 dx = b.x - a.x;
        vec3 ddx = normalize(dx);
        vec3 F = -(ks * (length(dx) - r) + kd * dot(b.v - a.v, ddx)) * ddx;
        a.addForce(-F);
        b.addForce(F);
    }
    void draw() const {
```

```

        JR::drawCylinder(a.x, b.x, 0.1, vec4(1, 0, 0, 1));
    }
};

struct Obstacle {
    virtual void resolveCollision(Particle& p) = 0;
    virtual void draw() = 0;
};

struct PlanarObstacle : Obstacle {
    vec3 p;
    vec3 N;
    float k = 0.2;
    PlanarObstacle(const vec3& pp, const vec3& NN) : p(pp), N(NN) {}
    virtual void resolveCollision (Particle& a) override {
        if (abs(dot(a.x - p, N)) < 0.001 && abs(dot(a.v, N)) < 0.001) {
            a.v = a.v - dot(a.v, N) * N;
            a.x = a.x - dot(a.x - p, N) * N;
        }
        else if (dot(a.x - p, N) < 0.001 && dot(a.v, N) < 0) {
            vec3 vN = dot(a.v, N) * N;
            a.v = a.v - vN - k * vN;
            a.x = a.x - dot(a.x - p, N) * N;
        }
    }
    virtual void draw() override {
        JR::drawQuad(p, N, vec2(1000), vec4(.3, .3, .3, 1));
    }
};

PlanarObstacle plane({ 0,0,0 }, { 0,1,0 });
Sphere sph(vec3(-5, 50, -5), 30);
std::vector<Particle> particles;
std::vector<Spring> springs;
const int DIV = 20;

void render() {
    plane.draw();
    sph.draw();
    for (const auto& p : particles) p.draw();
    for (const auto& s : springs) s.draw();
}

void frame(float dt) {
    const int frameDiv = 10;
    for (int i = 0; i < frameDiv; i++) {
        for (auto& p : particles) p.clearForce();
        for (auto& p : particles) p.addForce(p.m * G);
        for (auto& p : particles) p.addForce(-k_drag * p.v);
        for (auto& s : springs) s.applyForce();
        for (auto& p : particles) p.update(dt / frameDiv);
        for (auto& p : particles) plane.resolveCollision(p);
        for (auto& p : particles) p.resolveCollision(sph);
    }
}

bool key(int k) {
    if (k == '1') { particles[0].pinned = !particles[0].pinned; return true; }
    if (k == '2') { particles[DIV - 1].pinned = !particles[DIV - 1].pinned; return true; }
    return false;
}

void init() {
    particles.clear();
    springs.clear();

    for (int y = 0; y < DIV; y++) for (int x = 0; x < DIV; x++)
        particles.emplace_back(1, vec3(2 * x - 19, 119 - 2 * y, rand() / float(RAND_MAX) * 0.0001));

    for (int y = 0; y < DIV; y++) for (int x = 0; x < DIV - 1; x++)
        springs.emplace_back(particles[y * DIV + x], particles[y * DIV + x + 1]);
    for (int y = 0; y < DIV - 1; y++) for (int x = 0; x < DIV; x++)
        springs.emplace_back(particles[y * DIV + x], particles[(y + 1) * DIV + x]);

    for (int y = 0; y < DIV - 1; y++) for (int x = 0; x < DIV - 1; x++)
        springs.emplace_back(particles[y * DIV + x], particles[(y + 1) * DIV + x + 1]);
    for (int y = 0; y < DIV - 1; y++) for (int x = 0; x < DIV - 1; x++)
        springs.emplace_back(particles[y * DIV + x + 1], particles[(y + 1) * DIV + x]);

    for (int y = 0; y < DIV; y++) for (int x = 0; x < DIV - 2; x++)
        springs.emplace_back(particles[y * DIV + x], particles[y * DIV + x + 2]);
    for (int y = 0; y < DIV - 2; y++) for (int x = 0; x < DIV; x++)
        springs.emplace_back(particles[y * DIV + x], particles[(y + 2) * DIV + x]);
    particles[0].pinned = true;
    particles[DIV - 1].pinned = true;
}

int main(int argc, const char* argv[]) {

```

```
Window* window = new Window(800, 600, "Particle System");
window->alignment(align_t::ALL);
animView = new Simulation3DView(0, 0, 800, 600);
animView->renderFunc(render);
animView->frameCB(frame);
animView->initCB(init);
animView->keyCB(key);
window->show();
_JGL::run();
return 0;
```

```
}
```